



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Proyecto Bases de Datos

Sistema de Comandas Profesional

Materia: Bases de Datos

Profesor: Gabriel Hurtado Avilés

Alumnos:

Damas Garnica Adrian

Hernandez Barrios Samuel Rodrigo

Grupo: 3CV4

Carrera: Ingeniería en Sistemas Computacionales

Resumen

En este documento se recopila, de manera formal, el desarrollo de la base de datos del Sistema de Comandas Profesional realizado a lo largo de las prácticas de la materia de Bases de Datos, desde el análisis del caso de estudio y el diseño de sus modelos hasta su implementación, población y comprobación en PostgreSQL.

Abstract

This document formally compiles the development of the Sistema de Comandas Profesional database, carried out throughout the practices of the Databases course, from the analysis of the case study and the design of its models to its implementation, population, and verification in PostgreSQL.

Índice

1 Introducción 1
- 1.1 Planteamiento del problema 1
- 1.2 Justificación 1
- 1.3 Objetivo general 2
- 1.4 Objetivos específicos 2
- 1.5 Estado del arte 3
- 1.6 Alcance 3
- 1.7 Organización del documento 4

2 Marco teórico 5
- 2.1 Dato, información y base de datos 5
- 2.2 Sistema gestor de bases de datos 5
- 2.3 Ciclo de vida de una base de datos 6
- 2.4 Modelo entidad-relación y su extensión 7
- 2.5 Modelo relacional 8
- 2.6 Implementación en un gestor relacional 8
  - 2.6.1 Restricciones de dominio 8
  - 2.6.2 Integridad referencial 9
  - 2.6.3 Disparadores y vistas 9
  - 2.6.4 Transacciones 9
  - 2.6.5 Control de acceso 9
  - 2.6.6 Carga masiva de datos 10

3 Metodología y entorno de trabajo 11
- 3.1 Desarrollo por etapas 11
- 3.2 Control de versiones 11
- 3.3 Contenedores y gestor de bases de datos 12
- 3.4 Automatización y organización del repositorio 13

4 Caso de estudio y requerimientos 15
- 4.1 Descripción del caso 15
- 4.2 Levantamiento de requerimientos 15
- 4.3 Requerimientos de información 16
- 4.4 Requerimientos del sistema 17
- 4.5 Reglas del negocio 17

5 Diseño conceptual 19
- 5.1 Modelo entidad-relación inicial 19
- 5.2 Limitaciones del modelo básico 21
- 5.3 Modelo entidad-relación extendido 21
  - 5.3.1 Entidades débiles 23
  - 5.3.2 Jerarquía de Usuario 23
  - 5.3.3 Relaciones y cardinalidades 23
  - 5.3.4 Relación ternaria y atributos de relación 24

5.3.5 Atributos especiales . . . 25
5.4 Restricciones que el diagrama no expresa . . . 25

6 Diseño lógico . . . 26
6.1 Inventario del modelo extendido . . . 26
6.2 Tipos de dato . . . 26
6.3 Aplicación de las reglas de transformación . . . 27
6.3.1 Entidades, atributos compuestos y multivaluados . . . 27
6.3.2 Relaciones . . . 28
6.3.3 Entidades débiles . . . 28
6.3.4 Jerarquía de Usuario . . . 28
6.4 Esquema resultante . . . 29
6.5 Preservación del significado . . . 29

7 Implementación . . . 32
7.1 Ejecución del esquema y migraciones . . . 32
7.2 Tipos de dato y dominios . . . 32
7.3 Restricciones de columna . . . 33
7.4 Reglas del negocio con disparadores . . . 34
7.5 Vistas . . . 35
7.6 Usuarios y permisos . . . 35
7.7 Pruebas de las restricciones . . . 36

Índice de figuras

1 Flujo de ramas del repositorio, con ejemplos de las prácticas 4 y 5 . . . 12
2 Diagrama entidad-relación inicial . . . 20
3 Diagrama entidad-relación extendido en notación de Peter Chen . . . 22
4 Diagrama relacional del sistema de comandas . . . 30

Índice de tablas

1 Trabajos relacionados con el proyecto . . . 3
2 Características de un sistema gestor de bases de datos . . . 6
3 Etapas del ciclo de vida trabajadas en cada práctica . . . 11
4 Configuración del contenedor del gestor . . . 13
5 Organización del repositorio . . . 14
6 Hallazgos de la entrevista con el cliente . . . 16
7 Reglas que motivaron el modelo extendido . . . 21
8 Entidades débiles del modelo . . . 23
9 Relaciones del modelo entidad-relación extendido . . . 24
10 Inventario del modelo entidad-relación extendido . . . 26
11 Tipos de dato por clase de atributo . . . 27
12 Tablas del esquema relacional . . . 29
13 Migraciones aplicadas al esquema . . . 32
14 Ejemplos de restricciones `CHECK` . . . 34

15 Reglas del negocio implementadas con disparadores 34
16 Roles de la base de datos 36

Índice de códigos

1 Dominios de la migración `001-dominios.sql` 33
2 Función del disparador `tg_pago_saldo` 35

1 Introducción

En esta sección se presenta el problema que da origen al proyecto, las razones que justifican el diseño de una base de datos para atenderlo, los objetivos que guiaron su desarrollo, los trabajos relacionados que se revisaron y el alcance del documento.

1.1 Planteamiento del problema

Los salones de eventos que ofrecen servicio de banquetes gestionan la toma y el seguimiento de las comandas, es decir, los pedidos de alimentos y bebidas que realiza cada mesa durante un evento, con comandas de papel, hojas de cálculo o aplicaciones genéricas que no fueron diseñadas para este giro. Con esta forma de operar, la información de los pedidos se duplica o se pierde entre el área de meseros, la cocina y la caja; no queda un registro histórico confiable de lo que se sirvió en cada evento ni de su costo; el inventario de insumos se controla de manera aproximada, lo que provoca faltantes o compras innecesarias; y la cuenta final de cada evento se elabora conciliando a mano varias comandas, lo que la vuelve lenta y propensa a errores.

El salón que se tomó como caso de estudio atiende alrededor de quince eventos al mes, con unas veinte mesas por evento y dos o tres cuentas por mesa. Para atenderlo, el equipo contaba con un sistema de comandas desarrollado en un semestre anterior en la materia de Sistemas Operativos: una interfaz de terminal escrita en C que guardaba la información en archivos de texto. Ese sistema carecía de relaciones bien definidas, de normalización y de reglas de integridad, de modo que cualquier cambio en el negocio obligaba a modificar el programa y nada impedía que los archivos quedaran en un estado inconsistente.

Por lo anterior, el problema que se atiende no consiste solamente en digitalizar la toma de comandas, sino en diseñar e implementar, con una metodología formal, la base de datos que sustente una versión profesional de ese sistema, capaz de representar eventos, mesas, cuentas, comandas, productos, ingredientes, personal y pagos sin perder la consistencia de la información.

1.2 Justificación

El sistema de comandas constituye el núcleo operativo de un salón de eventos: de él dependen la asignación de mesas, el seguimiento de los pedidos, la disponibilidad de insumos y el cobro al cliente. Un modelo de datos mal estructurado produce inconsistencias que el personal descubre hasta el cierre del evento, cuando ya no es posible corregirlas, y limita la capacidad del sistema para adaptarse a nuevos requerimientos, como los paquetes contratados, las variantes de los productos o los reportes administrativos.

Desde el punto de vista académico, el proyecto permite aplicar de forma integral los conteni-

dos de la materia sobre un caso real y conocido por el equipo: el modelado entidad-relación y su versión extendida, la transformación al modelo relacional, la definición de datos con restricciones de dominio y de integridad referencial, el control de acceso y la manipulación de datos mediante transacciones. Contar con el sistema anterior como punto de comparación facilita identificar los errores de diseño que se cometieron y comprobar que el nuevo modelo los resuelve.

Desde el punto de vista práctico, una base de datos que hace cumplir por sí misma las reglas del negocio reduce los errores en la toma de comandas, permite conocer con precisión el consumo y las existencias de insumos y entrega cuentas correctas sin conciliación manual. Además, al concentrar las reglas en el gestor y no en el programa, cualquier aplicación que se conecte a la base, ya sea de escritorio, móvil o web, queda sujeta a las mismas validaciones.

1.3 Objetivo general

Diseñar, implementar y comprobar una base de datos relacional para el sistema de comandas de un salón de eventos, que gestione de manera íntegra y consistente la información de eventos, mesas, cuentas, comandas, productos, ingredientes, personal y pagos, y que sirva de sustento a la versión profesional del sistema desarrollado previamente.

1.4 Objetivos específicos

1. Identificar los requerimientos de información del salón de eventos mediante una entrevista con el cliente y fundamentar los conceptos de bases de datos que guían el diseño.
2. Construir el modelo entidad-relación del sistema y extenderlo con los conceptos del modelo entidad-relación extendido que el caso requiere.
3. Transformar el modelo extendido en un esquema relacional mediante las reglas de transformación y ejecutarlo en PostgreSQL dentro de un contenedor de Docker.
4. Declarar los tipos de dato, dominios, restricciones y permisos que protejan las reglas del negocio desde el propio gestor.
5. Poblar la base de datos a partir de una fuente documentada, mediante un script de carga repetible y eficiente.
6. Comprobar la integridad referencial y el comportamiento de las operaciones de manipulación de datos mediante pruebas y transacciones.

1.5 Estado del arte

Para ubicar el proyecto respecto de trabajos recientes se revisaron tres estudios relacionados con los modelos de datos, la elección de un sistema gestor y la seguridad de la información. La Tabla 1 resume cada uno y su relación con el proyecto.

Referencia	Problema y metodología	Resultados principales	Relación con el proyecto
Aguilar Vera et al. (2023)	Revisión sistemática de la literatura sobre el modelado y la gestión de bases de datos NoSQL, sus tipos de datos y sus herramientas.	Identifica cuatro modelos NoSQL: clave-valor, documentos, columnas y grafos; el orientado a documentos es el más flexible y el más usado.	Permite contrastar los modelos NoSQL con el modelo relacional que exigen las cuentas, los pagos y las comandas.
Taipalus (2024)	Revisión de 117 estudios que comparan el desempeño de 44 sistemas gestores, junto con su rigor metodológico y su reproducibilidad.	Hay pocas comparaciones rigurosas y reproducibles; en general, los gestores NoSQL superan a los NewSQL en lectura y escritura.	Sustenta que la elección del gestor considere la integridad y las transacciones, y no solo el desempeño.
Machuca & Braul (2022)	Revisión de catorce artículos publicados entre 2016 y 2021 sobre las metodologías de seguridad en bases de datos.	El control de acceso es la estrategia más utilizada, seguida del cifrado y de la protección de bases de datos en la nube.	Respalda la definición de roles y permisos distintos para la aplicación, la consulta y la administración.

Tabla 1
Trabajos relacionados con el proyecto

Los tres trabajos son revisiones sistemáticas y coinciden en que ninguna tecnología de bases de datos es superior en términos absolutos: la elección depende de las necesidades del sistema. Taipalus (2024) reporta que los gestores NoSQL superan a otras familias en velocidad de lectura y escritura; sin embargo, en un sistema de comandas una cuenta que no cuadra o un pago registrado dos veces afectan directamente la operación del salón, por lo que la integridad y la consistencia pesan más que la velocidad. Por esa razón se eligió PostgreSQL, un gestor relacional con soporte completo de transacciones y restricciones de integridad. De igual forma, lo reportado por Machuca y Braul (2022) sobre el control de acceso se tradujo en roles con el mínimo privilegio, de modo que cada usuario de la base pueda realizar únicamente las operaciones que le corresponden.

1.6 Alcance

El documento abarca la base de datos del sistema de comandas en todas sus etapas: el análisis del caso de estudio, el diseño conceptual con los modelos entidad-relación y entidad-relación extendido, el diseño lógico con el modelo relacional, la implementación física en PostgreSQL, la población con datos y las pruebas de integridad y de manipulación de datos. Cada etapa corresponde a una o varias de las prácticas de la materia, y su documentación detallada, junto con los scripts y las evidencias, se encuentra en el repositorio del equipo.

Quedan fuera de este documento la aplicación que utiliza la base de datos, cuyo cliente y servidor se desarrollan por separado, y el proyecto asignado por la materia, que se documenta de forma independiente.

1.7 Organización del documento

Después de esta introducción, el marco teórico reúne los conceptos que sustentan el diseño y la sección de metodología describe la forma de trabajo y el entorno en que se desarrolló el proyecto. Enseguida se presentan el caso de estudio con sus requerimientos, el diseño conceptual y el diseño lógico de la base de datos. Las secciones siguientes describen su implementación en PostgreSQL, la población con datos y las pruebas de integridad referencial y de manipulación de datos. Por último, se presentan los resultados y las conclusiones del proyecto.

2 Marco teórico

En esta sección se reúnen los conceptos que sustentan el diseño y la implementación de la base de datos: la noción de base de datos y de sistema gestor, las etapas de su ciclo de vida, los modelos con los que se representó el caso de estudio y los mecanismos del lenguaje SQL con los que el gestor hace cumplir las reglas del negocio.

2.1 Dato, información y base de datos

Un dato es un hecho que puede ser registrado y que tiene un significado implícito (Elmasri & Navathe, 2016, p. 4); por sí solo no tiene un significado más allá de su propia existencia (Bellinger et al., 2004, p. 1). La información surge cuando los datos se relacionan entre sí y adquieren significado a través de esa conexión (Bellinger et al., 2004, p. 1). Por ejemplo, el número de una mesa no dice nada aislado, pero ligado a un evento, a un mesero y a una cuenta permite saber quién atendió a qué grupo de invitados y cuánto consumió.

Una base de datos es, en consecuencia, una colección de datos relacionados de manera lógica y coherente, que representa algún aspecto del mundo real y que se diseña y se puebla con un propósito específico (Elmasri & Navathe, 2016, p. 5). Para Date (2001), es un sistema computarizado cuyo objetivo es llevar el registro de la información y permitir que los usuarios la consulten y la actualicen (p. 2).

2.2 Sistema gestor de bases de datos

Un sistema gestor de bases de datos (SGBD) es el conjunto de programas que facilita los procesos de definir, construir, manipular y compartir bases de datos entre diversos usuarios y aplicaciones (Elmasri & Navathe, 2016, p. 6). Su función es mediar entre los archivos físicos y quienes usan la información, de manera que las aplicaciones no dependan de la forma en que los datos se almacenan. Frente a los sistemas de archivos tradicionales, en los que cada programa define y mantiene sus propios archivos, un SGBD concentra la definición de los datos en un catálogo y ofrece las características que se resumen en la Tabla 2.

Característica	Descripción
Integración	Los datos se concentran en un solo repositorio en lugar de archivos independientes e inconexos.
Persistencia	Los datos sobreviven a los programas que los crearon y solo cambian por una petición al gestor.
Redundancia controlada	La duplicación se evita mediante la normalización y, cuando existe a propósito, el gestor mantiene las copias sincronizadas.
Integridad	Los datos cumplen las restricciones declaradas en el catálogo, que el gestor valida en cada actualización.
Independencia de datos	El esquema físico o lógico puede cambiar sin modificar los programas que acceden a los datos.
Seguridad	Solo los usuarios autorizados pueden leer o modificar cada objeto, según los privilegios que se les otorgan.
Concurrencia	Varios usuarios operan al mismo tiempo sin producir lecturas sucias ni actualizaciones perdidas.
Recuperación	Tras una falla, la base regresa a un estado consistente que conserva las transacciones confirmadas.

Tabla 2
Características de un sistema gestor de bases de datos

Estas características se ejercen mediante cuatro sublenguajes que, en los gestores basados en SQL, conviven en una misma interfaz (Elmasri & Navathe, 2016, pp. 38–41; Silberschatz et al., 2002, pp. 42–44): el lenguaje de definición de datos (DDL), con el que se crean tablas, dominios y restricciones; el lenguaje de manipulación de datos (DML), con el que se insertan, modifican, borran y consultan los datos; el lenguaje de control de datos (DCL), que administra los privilegios de los usuarios; y el lenguaje de control de transacciones (TCL), que delimita las unidades de trabajo que se confirman o se descartan por completo.

Para separar las aplicaciones de los detalles de almacenamiento, los gestores siguen la arquitectura de tres niveles propuesta por el comité ANSI/SPARC (Elmasri & Navathe, 2016, pp. 36–37): el nivel interno describe cómo se almacenan los datos, el nivel conceptual describe la estructura lógica de toda la base y el nivel externo ofrece vistas con la porción de datos que interesa a cada grupo de usuarios. Gracias a esta separación, un cambio en un nivel no obliga a modificar el nivel superior ni los programas de aplicación (Elmasri & Navathe, 2016, pp. 37–38).

2.3 Ciclo de vida de una base de datos

Una base de datos real tiene un ciclo de vida de muchos años, durante el cual debe evolucionar conforme cambian los requerimientos (Elmasri & Navathe, 2016, p. 8). Su desarrollo se divide en etapas que van de la idea del negocio a la operación, y cada una produce un entregable que sirve de entrada a la siguiente. Este proyecto siguió esas etapas, que se describen a continuación junto con la práctica en que se trabajó cada una.

1. **Recolección y análisis de requerimientos.** Se documenta qué información necesita la organización y qué operaciones realiza sobre ella (Elmasri & Navathe, 2016, p. 8). En el

proyecto corresponde a la entrevista con el cliente de la Práctica 1.

2. **Diseño conceptual.** Los requerimientos se expresan en un modelo semántico independiente del gestor, como el modelo entidad-relación (Chen, 1976, pp. 9–10). Se realizó en las prácticas 1 y 2.

3. **Diseño lógico.** El esquema conceptual se traduce al modelo del gestor elegido, por lo general el relacional, mediante reglas de transformación (Elmasri & Navathe, 2016, p. 289). Corresponde a la Práctica 3.

4. **Diseño físico e implementación.** El esquema lógico se escribe en sentencias DDL, se ejecuta en el gestor y la base se puebla con los datos iniciales (Date, 2001, p. 41; Elmasri & Navathe, 2016, p. 31). Se realizó en las prácticas 3 y 4.

5. **Pruebas y operación.** Se comprueba que la base proteja sus reglas y se pone en uso mediante las aplicaciones (Elmasri & Navathe, 2016, pp. 43–45). Las pruebas corresponden a las prácticas 4 y 5.

2.4 Modelo entidad-relación y su extensión

El modelo entidad-relación, propuesto por Chen (1976), representa el mundo real mediante entidades, que son los objetos de interés; atributos, que son sus propiedades; y relaciones, que son las asociaciones entre entidades. Un atributo puede ser simple o compuesto, monovaluado o multivaluado, y almacenado o derivado, cuando su valor se calcula a partir de otros datos (Elmasri & Navathe, 2016). Cada relación tiene un grado, que es el número de entidades que participan en ella, y restricciones estructurales que se expresan con la notación (min, max): el mínimo indica si la participación de una entidad es obligatoria y el máximo, con cuántas instancias de la otra entidad puede asociarse (Elmasri & Navathe, 2016).

Una entidad débil es aquella que no tiene una clave propia suficiente para identificarse y depende de una entidad propietaria a través de una relación identificadora; su clave se forma con la clave de la propietaria y una clave parcial (Elmasri & Navathe, 2016). Por ejemplo, la mesa número tres solo se distingue si se sabe a qué evento pertenece.

El modelo entidad-relación extendido (EER) añade los conceptos de especialización y generalización, con los que una entidad general, o superclase, se divide en subclases que heredan sus atributos y agregan los propios (Elmasri & Navathe, 2016). Una especialización está sujeta a dos restricciones: de disyunción, que indica si una instancia puede pertenecer a una sola subclase (disjunta) o a varias (solapada), y de completitud, que indica si toda instancia de la superclase debe pertenecer a alguna subclase (total) o no (parcial). El modelo extendido también contempla relaciones de grado superior, como las ternarias, que se justifican cuando ningún par de las entidades participantes determina a la tercera.

2.5 Modelo relacional

El modelo relacional representa la base de datos como una colección de relaciones, o tablas, en las que cada fila es una tupla y cada columna es un atributo cuyo valor pertenece a un dominio, es decir, a un conjunto de valores atómicos permitidos (Elmasri & Navathe, 2016). Una clave candidata es un conjunto mínimo de atributos que identifica de forma única a cada tupla; la clave primaria es la candidata que se elige para ese fin, y una clave foránea es un conjunto de atributos que hace referencia a la clave primaria de otra relación.

Sobre estas estructuras se definen las restricciones de integridad del modelo (Elmasri & Navathe, 2016): la restricción de dominio exige que cada valor pertenezca al dominio de su atributo; la de clave, que no existan dos tuplas con la misma clave; la de integridad de entidad, que la clave primaria nunca sea nula; y la de integridad referencial, que toda clave foránea haga referencia a una tupla que exista.

El paso del modelo extendido al relacional se realiza con un conjunto de reglas de transformación (Elmasri & Navathe, 2016, p. 289). Las reglas que se aplicaron en el proyecto convierten cada entidad fuerte en una tabla; descomponen los atributos compuestos en sus componentes; llevan cada atributo multivaluado a una tabla propia; transforman las relaciones uno a muchos en una clave foránea del lado de muchos, las de muchos a muchos en una tabla intermedia y las de uno a uno en una clave foránea única; convierten las relaciones de grado superior en una tabla cuya clave combina las claves de las entidades participantes; forman la clave de cada entidad débil con la de su propietaria; y representan las jerarquías con una de varias estrategias, como una tabla por superclase y una por subclase.

2.6 Implementación en un gestor relacional

Los conceptos anteriores se hacen cumplir en el gestor mediante las sentencias de SQL. En esta sección se describen los mecanismos de PostgreSQL que se utilizaron en el proyecto.

2.6.1 Restricciones de dominio

Al definir una tabla, cada columna recibe un tipo de dato y, opcionalmente, restricciones que acotan sus valores (PostgreSQL Global Development Group, s.f.-a): `NOT NULL` impide que la columna quede vacía; `DEFAULT` asigna un valor inicial cuando no se especifica uno; `UNIQUE` impide que dos filas repitan el mismo valor o combinación de valores; y `CHECK` exige que cada fila cumpla una condición, ya sea un rango, una lista de valores permitidos, un formato o una comparación entre columnas. Cuando un mismo tipo y una misma precisión se repiten en varias tablas, la sentencia `CREATE DOMAIN` permite definirlos una sola vez con un nombre propio y reutilizarlos (PostgreSQL Global Development Group, s.f.-c).

2.6.2 Integridad referencial

Una clave foránea se declara con `FOREIGN KEY` y hace que el gestor rechace cualquier fila que apunte a un renglón inexistente (PostgreSQL Global Development Group, s.f.-a). La declaración incluye además una acción referencial, que determina qué ocurre con las filas hijas cuando se borra o se actualiza la fila padre: `RESTRICT` impide la operación mientras existan hijas; `CASCADE` la propaga a las hijas; `SET NULL` deja la referencia en nulo; y `SET DEFAULT` la reemplaza por un valor por omisión. La elección depende de la regla del negocio: `CASCADE` corresponde cuando la hija no tiene sentido sin su padre, como una entidad débil, y `RESTRICT` cuando el padre no debe desaparecer mientras algo dependa de él.

2.6.3 Disparadores y vistas

Algunas reglas del negocio relacionan varias filas o varias tablas y no pueden expresarse con una restricción de columna. Para ellas se emplean disparadores, que son funciones que el gestor ejecuta automáticamente antes o después de una inserción, actualización o borrado, y que pueden rechazar la operación o modificar los datos (PostgreSQL Global Development Group, s.f.-i). Por su parte, una vista es una consulta almacenada con un nombre, que se consulta como si fuera una tabla (PostgreSQL Global Development Group, s.f.-d); se utiliza para calcular atributos derivados sin guardarlos, de modo que su valor nunca queda desactualizado, y para ofrecer a cada grupo de usuarios solo la porción de datos que necesita.

2.6.4 Transacciones

Una transacción es una unidad de trabajo que agrupa varias operaciones y que debe ejecutarse completa o no ejecutarse (PostgreSQL Global Development Group, s.f.-h). Las transacciones cumplen las propiedades conocidas como ACID: atomicidad, porque sus operaciones se aplican todas o ninguna; consistencia, porque llevan la base de un estado válido a otro; aislamiento, porque su ejecución no se ve afectada por otras transacciones concurrentes; y durabilidad, porque sus cambios confirmados persisten aun ante fallas (Elmasri & Navathe, 2016). En SQL, una transacción inicia con `BEGIN`, se confirma con `COMMIT` y se descarta con `ROLLBACK`. Dentro de ella, `SAVEPOINT` marca un punto de recuperación al que es posible regresar con `ROLLBACK TO` sin descartar el trabajo anterior (PostgreSQL Global Development Group, s.f.-g).

2.6.5 Control de acceso

El acceso a la base se administra mediante roles, que pueden representar a un usuario o a un grupo de usuarios (PostgreSQL Global Development Group, s.f.-e). Con `GRANT` se otorgan privilegios sobre cada objeto, como leer, insertar, modificar o borrar en una tabla, y con `REVOKE` se retiran (PostgreSQL Global Development Group, s.f.-f). El principio de mínimo privilegio con-

siste en conceder a cada rol únicamente los permisos que su función requiere, de modo que un error en una aplicación o un acceso indebido no pueda afectar más datos de los necesarios.

2.6.6 Carga masiva de datos

Para poblar una base con volúmenes grandes de datos, insertar cada fila con una sentencia `INSERT` resulta lento, porque cada sentencia se procesa por separado. La sentencia `COPY` de PostgreSQL, y su variante `\copy` del cliente `psql`, transfiere el contenido completo de un archivo a una tabla en una sola operación (PostgreSQL Global Development Group, s.f.-b), por lo que es el mecanismo recomendado para la carga inicial de una base de datos.

3 Metodología y entorno de trabajo

En esta sección se describe la forma en que se organizó el desarrollo del proyecto, las herramientas con las que el equipo coordinó su trabajo y el entorno en que se ejecutó el gestor de bases de datos.

3.1 Desarrollo por etapas

El proyecto se desarrolló siguiendo las etapas del ciclo de vida de una base de datos descritas en el marco teórico. Cada práctica de la materia correspondió a una o varias de esas etapas, y su resultado fue la entrada de la práctica siguiente, como se resume en la Tabla 3.

Práctica	Etapa	Productos
1	Requerimientos y diseño conceptual	Marco teórico, entrevista con el cliente, requerimientos, diagrama entidad-relación y entorno de trabajo con Git y Docker.
2	Diseño conceptual	Modelo entidad-relación extendido en notación de Peter Chen y análisis de los artículos del proyecto asignado.
3	Diseño lógico e implementación	Inventario del modelo extendido, reglas de transformación, esquema relacional ejecutado en PostgreSQL, diccionario de datos y pruebas de las restricciones.
4	Implementación y población	Tipos de dato y dominios, restricciones de columna, fuente de datos, script de carga, pruebas de dominio, usuarios y permisos.
5	Pruebas de integridad	Claves foráneas y acciones referenciales, pruebas de integridad, edición de valores, transacciones y experimentos individuales.

Tabla 3
Etapas del ciclo de vida trabajadas en cada práctica

El desarrollo no fue estrictamente lineal. Cada etapa permitió detectar omisiones de la anterior y corregirlas antes de continuar: al construir el modelo extendido se reconsideraron los requerimientos de la entrevista; al transformar el modelo al relacional se levantó primero un inventario de entidades, relaciones y atributos especiales; y al implementar las restricciones de dominio se encontraron reglas que el esquema todavía no protegía. A partir de la Práctica 4, todo cambio al esquema ya ejecutado se registró como una migración numerada con sentencias `ALTER TABLE`, en lugar de editar el script original, de modo que la historia del esquema quedara documentada y pudiera reproducirse en cualquier base.

3.2 Control de versiones

El código, la documentación y las evidencias del proyecto se administran con Git en un repositorio compartido en GitHub. Git registra cada cambio como una confirmación con su autor y su

fecha, lo que permite reconstruir la historia del proyecto y recuperar cualquier versión anterior (Chacon & Straub, 2014); GitHub agrega un repositorio remoto común y la revisión de cambios mediante *pull requests* (GitHub, s.f.-b).

El equipo adoptó un flujo de trabajo basado en ramas (Atlassian, s.f.), representado en la Figura 1. La rama `main` contiene únicamente el trabajo revisado y entregado. Al iniciar una práctica se crea a partir de ella una rama de práctica, y cada sección de la práctica se desarrolla en una rama propia que sale de la rama de práctica. Al terminar una sección, sus cambios se integran a la rama de práctica mediante un *pull request*, y al cerrar la práctica, la rama de práctica se integra a `main` con un *pull request* revisado por otro integrante del equipo.

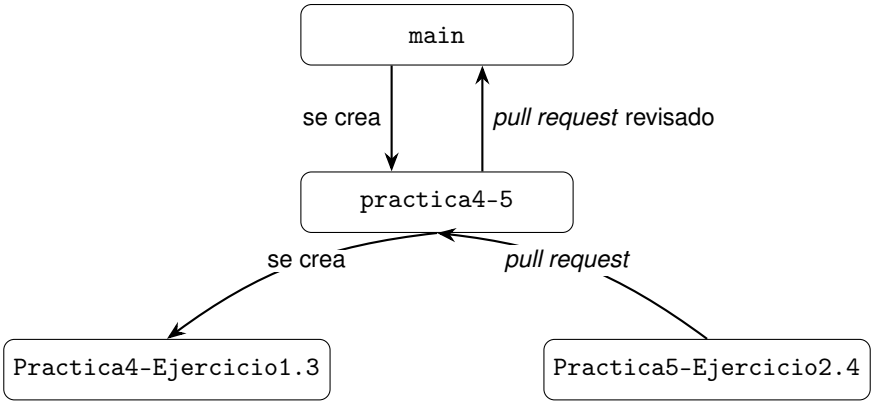


Figura 1
Flujo de ramas del repositorio, con ejemplos de las prácticas 4 y 5

La revisión entre pares antes de cada integración permite detectar errores, proteger la rama principal y compartir el conocimiento entre los integrantes (GitHub, s.f.-a). A lo largo del proyecto se integraron cerca de cuarenta *pull requests*, y cada integrante registró sus propias confirmaciones, de modo que el historial del repositorio funciona también como evidencia de la participación de cada uno.

3.3 Contenedores y gestor de bases de datos

El gestor de bases de datos se ejecuta en un contenedor de Docker. Un contenedor empaqueta una aplicación con todo lo que necesita para funcionar y la aísla del sistema anfitrión, de manera que se comporta igual en cualquier equipo (Docker, s.f.-c). Esto resuelve un problema habitual en el trabajo en equipo: que la base funcione en la computadora de un integrante y no en la de otro por diferencias de versión o de configuración.

La definición del contenedor se encuentra en un archivo de Docker Compose que forma parte del repositorio y que se ha reutilizado sin cambios desde la Práctica 1. Sus parámetros principales se presentan en la Tabla 4.

Parámetro	Valor
Gestor	PostgreSQL 16.4, sobre la imagen oficial `postgres:16.4-alpine`
Contenedor	`practica1-postgres`
Puerto publicado	`5432` en el anfitrión y en el contenedor
Volumen	`practica1-pgdata`, montado en el directorio de datos del gestor
Codificación	UTF-8
Zona horaria	`America/Mexico_City`
Credenciales	Variables de entorno, con valores por omisión para el entorno de desarrollo

Tabla 4
Configuración del contenedor del gestor

Los datos de un contenedor se pierden cuando este se elimina, salvo que se guarden en un volumen, que es un espacio de almacenamiento administrado por Docker e independiente del ciclo de vida del contenedor (Docker, s.f.-b). Por ello, el directorio de datos de PostgreSQL se monta en un volumen con nombre. En la Práctica 1 se comprobó que la base y sus datos se conservan después de eliminar y volver a crear el contenedor. El puerto publicado permite, además, que las herramientas del equipo anfitrión, como el cliente `psql` o un administrador gráfico, se conecten al gestor como si estuviera instalado localmente (Docker, s.f.-a).

3.4 Automatización y organización del repositorio

Para que cualquier integrante pudiera reproducir el estado de la base y verificar que las reglas del esquema se cumplen, las tareas repetitivas se automatizaron con un archivo `Makefile`. Un solo comando levanta el contenedor, recrea una base de pruebas, ejecuta el esquema, las migraciones y los datos de ejemplo, y corre todas las pruebas: las instrucciones que deben ser rechazadas, los casos válidos y las pruebas de integridad referencial. Si alguna prueba no da el resultado esperado, el proceso se detiene e informa cuál falló. La carga de la base completa, las pruebas de permisos y los ejercicios de manipulación de datos cuentan con sus propios scripts, que leen las credenciales de variables de entorno y nunca las guardan en el repositorio.

El repositorio se organiza por tipo de contenido, como se muestra en la Tabla 5. Además, desde la carpeta `web` se publica un sitio del proyecto mediante GitHub Pages.

Carpeta	Contenido
`docs`	Documentación de cada práctica en Markdown, enunciados y este documento en LaTeX
`sql`	Esquema, migraciones, pruebas, script de carga, datos y sentencias de la Práctica 5
`modelo`	Diagramas entidad-relación, entidad-relación extendido y relacional
`evidencias`	Capturas y salidas de la terminal de cada práctica
`entorno`	Definición del contenedor del gestor
`web`	Sitio del proyecto

Tabla 5
Organización del repositorio

4 Caso de estudio y requerimientos

En esta sección se describe el negocio para el que se diseñó la base de datos, la forma en que se obtuvieron sus requerimientos y las reglas que la base debe hacer cumplir.

4.1 Descripción del caso

El primer cliente del sistema es un salón de eventos que ofrece servicio de banquetes. El salón atiende un solo evento a la vez y alrededor de quince eventos al mes; en cada evento se instalan cerca de veinte mesas, numeradas, y los invitados de cada mesa suelen dividirse en dos o tres grupos que pagan por separado. Cada grupo realiza entre uno y tres pedidos durante el evento, aunque puede pedir más veces.

La operación actual se apoya en comandas de papel. Los meseros toman los pedidos en las mesas y los entregan al responsable de caja, quien los reenvía a la cocina o al bar y es el único que puede cancelarlos o modificarlos. El organizador del evento puede contratar con anticipación un paquete de comida preparada, que no se cobra aparte, mientras que el consumo adicional a la carta se cobra a cada grupo. El salón recibe anticipos antes del evento y acepta pagos en efectivo, por transferencia o con tarjeta.

Para este giro, el equipo contaba con un sistema de comandas desarrollado en la materia de Sistemas Operativos: una interfaz de terminal escrita en C con la biblioteca `ncurses`, que guardaba la información en archivos de texto. Ese sistema sirvió como punto de partida para conocer la operación, pero su forma de almacenar los datos es justamente lo que el proyecto busca sustituir.

4.2 Levantamiento de requerimientos

Los requerimientos se obtuvieron mediante una entrevista con el cliente, organizada en siete áreas: eventos y mesas, menú y productos, flujo de comandas, cobros y pagos, roles y permisos, reportes, y operación y escalabilidad. La Tabla 6 resume los hallazgos de cada área; las preguntas y respuestas completas se encuentran en el repositorio del proyecto.

Área	Hallazgos principales
Eventos y mesas	Un evento a la vez, con alrededor de veinte mesas numeradas cuya cantidad varía. Cuando hay varios meseros, a cada uno se le asigna un grupo de mesas.
Menú y productos	Alimentos, bebidas con y sin alcohol, y dulcería, con precio fijo durante el evento y variantes de tamaño o preparación. Cada producto tiene una receta, y el salón necesita saber cuánto queda de cada ingrediente y cuándo resurtir.
Flujo de comandas	Los pedidos se centralizan en caja, pasan por los estados pendiente, en preparación, lista y entregada, y pueden cancelarse. Una cuenta puede pedir varias rondas y cada pedido admite notas especiales.
Cobros y pagos	El cobro más común es por grupo de invitados. Hay anticipos, tres formas de pago y no se requiere factura fiscal. La comida del paquete no se cobra aparte; el consumo a la carta sí.
Roles y permisos	Existen los roles de mesero, cocina, bar y cajero o administrador, y es necesario saber qué mesero levantó cada comanda y qué cuentas atiende.
Reportes	Ventas e ingresos por evento, y una comparación entre el consumo real y lo presupuestado en el paquete, consultados al cierre y también durante el evento.
Operación	Alrededor de quince eventos al mes, de preferencia sin depender de una conexión a internet y sin planes de abrir otra sucursal.

Tabla 6
Hallazgos de la entrevista con el cliente

En la Práctica 2, al construir el modelo extendido, se revisaron de nuevo las respuestas de la entrevista. Esa revisión mostró que varias reglas que el cliente sí había mencionado no podían expresarse con el modelo entidad-relación básico y habían quedado aplanadas o fuera del diagrama: que los números de mesa se reinician en cada evento, que cada tipo de usuario guarda datos distintos, que un evento puede tener varios organizadores, que lo acordado de cada producto cambia de un evento a otro aunque se contrate el mismo paquete, y que el inventario debe llevarse por ingrediente y no como un campo del producto. Estos requerimientos se incorporaron al diseño conceptual que se describe en la sección siguiente.

4.3 Requerimientos de información

A partir de la entrevista se identificaron los datos que la base debe almacenar, que se agrupan en cuatro ámbitos:

- **Eventos:** cada evento, con su fecha, tipo, número de asistentes, estado y organizadores; las mesas que se instalan en él; los paquetes que se contratan y la cantidad acordada de cada producto; y los anticipos que se reciben antes de su fecha.
- **Catálogo e inventario:** los productos, con su categoría, descripción, precio y variantes; los ingredientes, con su unidad de medida, existencia actual y existencia mínima; y la receta que indica cuánto de cada ingrediente lleva una porción de cada producto.
- **Operación:** las cuentas de cada mesa, con el nombre que las distingue entre sí; las

comandas que genera cada cuenta, con su producto, precio, estado, notas, el mesero que las levantó, la estación que las prepara y, en su caso, el cajero que las canceló.

- **Personal y cobro:** los usuarios del sistema con su rol, el turno de cada mesero y la estación de cocina o bar; y los pagos de cada cuenta, con su monto, forma de pago, fecha y la propina asignada al mesero.

4.4 Requerimientos del sistema

Además de los datos, la entrevista estableció lo que el sistema debe permitir hacer con ellos. Los requerimientos funcionales son los siguientes:

1. Registrar cada evento con sus mesas y asignar uno o varios meseros a ellas.
2. Permitir varias cuentas independientes por mesa, cada una con su propio consumo y su propio cobro.
3. Registrar las comandas de cada cuenta, en varias rondas, con notas especiales cuando hagan falta, y dar seguimiento a su estado hasta su entrega o cancelación.
4. Restringir la cancelación y la modificación de las comandas al responsable de caja.
5. Registrar la receta de cada producto, descontar del inventario los ingredientes que consume cada comanda y avisar cuando un ingrediente quede por debajo de su existencia mínima.
6. Registrar anticipos por evento y pagos por cuenta, con su forma de pago y la propina de cada mesero.
7. Distinguir el consumo incluido en el paquete del evento del consumo adicional a la carta.
8. Diferenciar los accesos de meseros, cocina, bar y caja según su función.
9. Generar reportes de ventas e ingresos por evento y comparar el consumo real con lo presupuestado.

Como requerimientos no funcionales, el sistema debe operar dentro del salón sin depender de una conexión a internet, soportar el volumen descrito en el caso con margen para crecer, y emitir únicamente comprobantes internos, ya que el salón no requiere facturación fiscal.

4.5 Reglas del negocio

De los requerimientos anteriores se desprenden las reglas que la base de datos debe proteger por sí misma, sin depender de que la aplicación las respete. Las principales son las siguientes:

- Una mesa y una cuenta solo se identifican dentro de su evento: la mesa tres de un evento

y la de otro son mesas distintas.

- Toda mesa tiene un mesero responsable, aunque un mesero puede no tener mesas asignadas.
- Una comanda solo avanza en el orden pendiente, en preparación, lista y entregada; puede cancelarse en cualquier momento, pero la cancelación debe registrar qué cajero la hizo.
- El precio que se cobra en una comanda es el del catálogo en el momento del pedido y no puede alterarse después.
- Cada comanda se prepara en la estación que corresponde a su producto: las bebidas en el bar y los alimentos y la dulcería en la cocina.
- Las existencias de un ingrediente nunca pueden quedar por debajo de cero.
- No se pueden levantar comandas en una cuenta cerrada ni en un evento que ya terminó.
- Un pago no puede exceder lo que se debe en la cuenta, la propina no puede superar el monto pagado y una cuenta solo puede cerrarse sin adeudo y con todas sus comandas entregadas o canceladas.

Algunas de estas reglas se precisaron durante el desarrollo, al revisar el diseño o al probar el esquema, pero todas tienen su origen en la operación que el cliente describió. Las secciones de diseño e implementación explican cómo se representa y se hace cumplir cada una.

5 Diseño conceptual

En esta sección se presenta el diseño conceptual de la base de datos, que se construyó en dos momentos: un modelo entidad-relación básico, elaborado a partir de la entrevista en la Práctica 1, y su versión extendida, elaborada en la Práctica 2 para representar las reglas que el modelo básico no podía expresar.

5.1 Modelo entidad-relación inicial

El primer modelo identificó los objetos del negocio sobre los cuales el salón necesita guardar información: el evento, la mesa, la cuenta de cada grupo de invitados, la comanda, el producto, el ingrediente, el pago, el anticipo, el paquete contratado y el usuario que opera el sistema. Cada uno se definió como entidad porque tiene datos propios y participa en reglas de la operación, y cada asociación que el cliente describió se representó como una relación con su razón de cardinalidad. La Figura 2 muestra el diagrama resultante.

En este modelo, la comanda ocupa el lugar central: sustituye a la comanda de papel, se liga a la cuenta que la generó, al producto que se pidió y al personal que la atendió, y registra su estado. Separar la cuenta de la mesa permite cobrar por grupo de invitados, y separar el anticipo del pago distingue los depósitos del organizador, ligados al evento, de los cobros que se hacen a cada cuenta durante el evento.

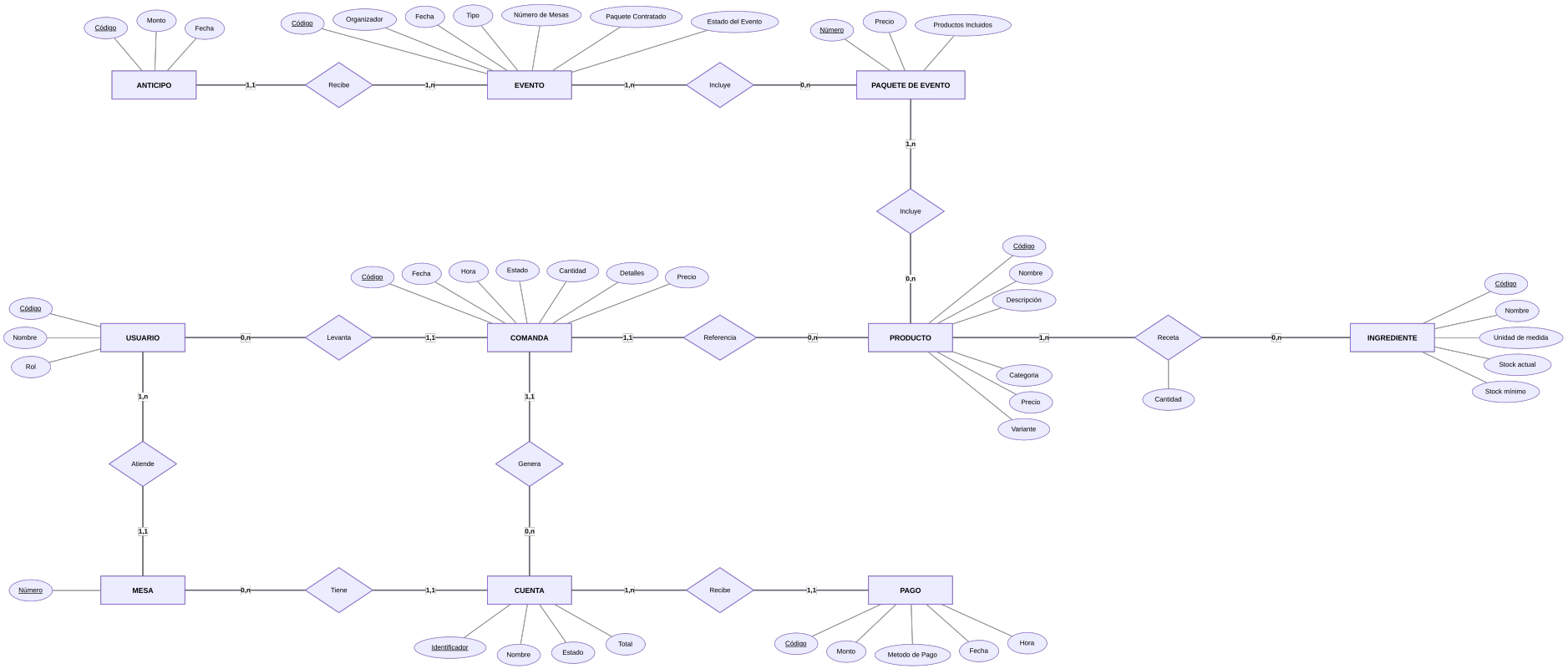


Figura 2
Diagrama entidad-relación inicial

5.2 Limitaciones del modelo básico

Al revisar de nuevo la entrevista en la Práctica 2, se encontró que el modelo básico obligaba a aplanar o a omitir varias reglas que el cliente sí había mencionado. La Tabla 7 resume esas reglas y el concepto del modelo extendido con el que se representaron.

Regla del negocio	Modelo básico	Modelo extendido
Los números de mesa y de cuenta se reinician en cada evento.	Mesa y Cuenta con clave propia.	Entidades débiles identificadas por el evento.
Meseros, cocina y bar, y caja guardan datos distintos y realizan funciones distintas.	Un atributo *rol* en Usuario.	Jerarquía de especialización de Usuario.
Ninguna mesa queda sin mesero, pero un mesero puede no tener mesas.	Razón 1:N, sin indicar la obligatoriedad.	Cardinalidades (min, max).
Lo acordado de cada producto cambia de un evento a otro aunque sea el mismo paquete.	Solo el contenido general del paquete.	Relación ternaria *Acuerda* con su cantidad.
Un evento puede tener varios organizadores.	Un solo valor por atributo.	Atributo multivaluado.
La fecha y la hora se consultan por día, mes y año.	Campos sueltos.	Atributos compuestos.

Tabla 7
Reglas que motivaron el modelo extendido

5.3 Modelo entidad-relación extendido

El modelo extendido conserva las entidades del modelo inicial y les agrega los conceptos de la Tabla 7. Se representó en la notación de Peter Chen, en la que las entidades se dibujan como rectángulos, las relaciones como rombos y los atributos como óvalos; las entidades débiles y sus relaciones identificadoras se distinguen con doble contorno. La Figura 3 muestra el diagrama completo.

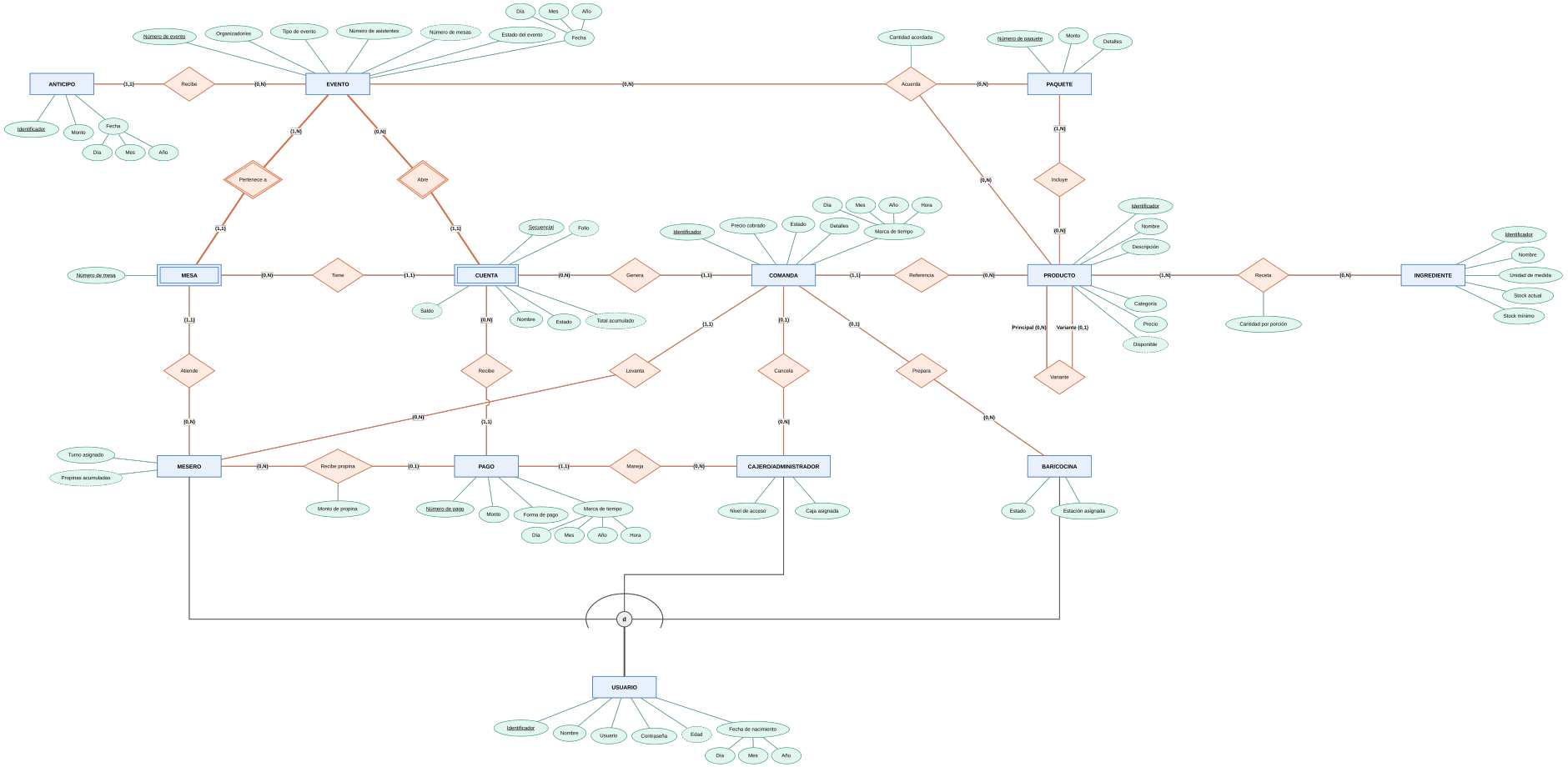


Figura 3
Diagrama entidad-relación extendido en notación de Peter Chen

5.3.1 Entidades débiles

Mesa y Cuenta son entidades débiles con dependencia de identificación respecto del evento, como se resume en la Tabla 8. Decir «la mesa tres» o «la cuenta diez» no identifica nada si no se indica el evento, porque la numeración se reinicia en cada uno. Por la misma razón, ninguna de las dos puede existir sin el evento al que pertenece.

Entidad débil	Propietaria	Clave parcial	Clave completa
Mesa	Evento	Número de mesa	(Número de evento, Número de mesa)
Cuenta	Evento	Secuencial	(Número de evento, Secuencial)

Tabla 8
Entidades débiles del modelo

5.3.2 Jerarquía de Usuario

Usuario se especializa en tres subclases: Mesero, Bar/Cocina y Cajero/Administrador. La especialización se justifica porque cada subclase tiene atributos propios, como el turno del mesero o la estación asignada de la cocina o el bar, y sobre todo relaciones propias: el mesero atiende mesas y levanta comandas, la cocina y el bar preparan comandas, y el cajero maneja pagos y cancela comandas. Si solo cambiaran los atributos, un campo de rol habría bastado.

La especialización es **disjunta**, porque una persona desempeña un solo puesto a la vez, y esa separación es la que da sentido al control que pidió el cliente: quien cobra no puede ser quien recibe la propina de ese cobro, y quien cancela una comanda no puede ser quien la levantó. Es además **total**, porque todo usuario del sistema es personal del salón y tiene un puesto; el organizador del evento no es usuario, sino un dato del evento. La cocina y el bar se agruparon en una sola subclase porque tienen los mismos atributos y la misma función, y su estación asignada indica cuál es cuál.

5.3.3 Relaciones y cardinalidades

Las cardinalidades se expresaron con la notación (min, max), que indica para cada entidad con cuántas instancias de la otra se asocia como mínimo y como máximo. La Tabla 9 reúne las diecisiete relaciones del modelo; en cada par, la primera cardinalidad corresponde a la primera entidad.

Relación	Entidades	Cardinalidad	Regla
Pertenece (identificadora)	Mesa – Evento	(1,1):(1,N)	Toda mesa es de un evento y todo evento tiene mesas.
Abre (identificadora)	Evento – Cuenta	(0,N):(1,1)	Toda cuenta se abre en un evento.
Atiende	Mesa – Mesero	(1,1):(0,N)	Ninguna mesa queda sin mesero.
Tiene	Mesa – Cuenta	(0,N):(1,1)	Una mesa tiene varias cuentas.
Genera	Cuenta – Comanda	(0,N):(1,1)	Una cuenta pide varias rondas.
Levanta	Mesero – Comanda	(0,N):(1,1)	Cada comanda tiene un mesero responsable.
Prepara	Bar/Cocina – Comanda	(0,N):(0,1)	Una comanda pendiente aún no tiene estación.
Cancela	Cajero – Comanda	(0,N):(0,1)	Solo caja cancela comandas.
Referencia	Comanda – Producto	(1,1):(0,N)	Cada comanda pide un producto.
Variante	Producto – Producto	(0,N):(0,1)	Una variante tiene un solo producto principal.
Receta	Producto – Ingrediente	(1,N):(0,N)	Todo producto lleva al menos un ingrediente.
Incluye	Paquete – Producto	(1,N):(0,N)	Contenido base de cada paquete.
Acuerda (ternaria)	Evento – Paquete – Producto	(0,N) en cada una	Lo pactado para cada evento.
Recibe	Evento – Anticipo	(0,N):(1,1)	Los anticipos son del evento.
Recibe	Cuenta – Pago	(0,N):(1,1)	Una cuenta puede pagarse en partes.
Maneja	Cajero – Pago	(0,N):(1,1)	Cada pago lo registra un cajero.
Recibe propina	Mesero – Pago	(0,N):(0,1)	Un pago puede dejar propina a un mesero.

Tabla 9
Relaciones del modelo entidad-relación extendido

5.3.4 Relación ternaria y atributos de relación

Una relación entre tres entidades solo se justifica cuando ninguna de ellas queda determinada por las otras dos. *Acuerda* cumple esa condición: si un evento contrata dos paquetes que incluyen el mismo producto, conocer el evento y el producto no indica cuánto se acordó por cada paquete, por lo que se necesitan las tres entidades. En cambio, *Prepara* se mantuvo binaria, porque cada comanda apunta a un solo producto y agregarlo a la relación no aportaba información.

Tres datos no pertenecen a ninguna entidad, sino a la relación que las une: la *cantidad acordada* de *Acuerda*, base de la comparación entre lo presupuestado y lo consumido; la *cantidad por porción* de *Receta*, con la que se descuenta el inventario al preparar cada comanda; y el *monto de propina* de *Recibe propina*, que corresponde a un cobro concreto y no al mesero ni al pago por separado.

5.3.5 Atributos especiales

El modelo incluye un atributo multivaluado, los organizadores del evento, y atributos compuestos para la fecha y la hora, que se consultan por sus componentes. También identifica atributos derivados, cuyo valor se calcula a partir de otros datos y no se captura: el total acumulado de cada cuenta, las propinas acumuladas de cada mesero, la edad de cada usuario, el número de mesas de cada evento y la disponibilidad de cada producto, que depende de que todos los ingredientes de su receta alcancen para una porción.

5.4 Restricciones que el diagrama no expresa

Un diagrama entidad-relación no puede representar todas las reglas del negocio. Las siguientes quedaron documentadas junto al modelo para resolverse en la implementación:

- El total acumulado de una cuenta debe coincidir con la suma de sus comandas.
- El precio cobrado se copia del catálogo al levantar la comanda y no cambia aunque después cambie el precio del producto.
- Solo el cajero o administrador puede cancelar comandas.
- Al preparar una comanda se descuenta del inventario la cantidad por porción de cada ingrediente, y la existencia nunca puede quedar negativa.
- El rol que se guarda es el actual: si un mesero pasa a caja, las comandas y los pagos que ya registró conservan su autoría.
- El contenido de un paquete es una base y no una lista cerrada, por lo que en un evento pueden acordarse productos o cantidades distintos de los del catálogo.

La sección de implementación describe cómo cada una de estas reglas se convirtió en una restricción, un disparador o una vista del esquema.

6 Diseño lógico

En esta sección se describe la transformación del modelo entidad-relación extendido en un esquema relacional, trabajada en la Práctica 3. Se presentan el inventario del que partió la transformación, las decisiones de tipos de dato, la forma en que se aplicó cada regla, el esquema resultante y la revisión de que este conserva el significado del modelo.

6.1 Inventario del modelo extendido

Antes de aplicar las reglas se levantó un inventario de los elementos del modelo extendido, porque cada tipo de elemento se transforma con una regla distinta y omitir uno habría dejado una parte del negocio sin representar. La Tabla 10 resume ese inventario.

Elemento	Cantidad
Entidades fuertes	8
Entidades débiles	2
Subclases de la jerarquía de Usuario	3
Relaciones binarias uno a muchos	14
Relaciones binarias muchos a muchos	2
Relaciones ternarias	1
Atributos multivaluados	1
Atributos compuestos	5
Atributos derivados	7
Atributos de relación	3

Tabla 10
Inventario del modelo entidad-relación extendido

6.2 Tipos de dato

Para no justificar cada columna por separado, se fijó primero el tipo que recibe cada clase de atributo, como se muestra en la Tabla 11. El criterio general fue elegir el tipo que valida el dato por sí mismo: un tipo de fecha rechaza el 31 de febrero y un decimal exacto evita que las sumas de dinero acumulen errores de redondeo, como ocurriría con los tipos de punto flotante.

Clase de atributo	Tipo en PostgreSQL	Razón
Identificadores	`INTEGER` con identidad	El gestor asigna el siguiente número y permite cargar números conocidos.
Claves parciales	`SMALLINT`	Se reinician en cada evento y nunca son grandes.
Dinero	`NUMERIC(10,2)`	Decimal exacto a centavos.
Cantidades de inventario	`NUMERIC(10,3)`	Decimal exacto en gramos, mililitros o piezas.
Listas cerradas	`VARCHAR` con `CHECK`	La lista queda visible y puede ampliarse.
Fechas e instantes	`DATE` y `TIMESTAMPTZ`	Validan la fecha y guardan el instante exacto.
Contraseñas	`VARCHAR(255)`	Solo se guarda su *hash*, nunca el texto.

Tabla 11
Tipos de dato por clase de atributo

6.3 Aplicación de las reglas de transformación

La transformación siguió las nueve reglas descritas en el marco teórico (Elmasri & Navathe, 2016, p. 289). En algunos casos fue necesario interpretar la regla para no perder una propiedad del modelo; esas decisiones se explican a continuación.

6.3.1 Entidades, atributos compuestos y multivaluados

Las ocho entidades fuertes produjeron ocho tablas, cada una con su identificador como clave primaria. Los cinco atributos compuestos son todos de fecha u hora, y el cliente los consulta por sus componentes; aplicar la regla al pie de la letra, guardando el día, el mes y el año como columnas sueltas, habría permitido guardar fechas imposibles. Por ello, la fecha se guarda con el tipo nativo del gestor, que la valida, y cada componente es una columna generada que el gestor calcula y que nadie puede escribir, de modo que nunca contradice a la fecha. El único atributo multivaluado, los organizadores del evento, se llevó a una tabla propia cuya clave combina el evento y el organizador.

Los atributos derivados no se guardan, porque una copia de un dato calculado debe mantenerse sincronizada y, si alguna vez no se actualiza, la base tiene dos versiones distintas de lo mismo. En su lugar, se calculan en vistas: el número de mesas del evento, el total acumulado y el saldo de cada cuenta, la edad de cada usuario, las propinas acumuladas de cada mesero y la disponibilidad de cada producto. El precio cobrado de una comanda, en cambio, sí se guarda, porque es el precio que tenía el producto al momento del pedido y no debe cambiar aunque el catálogo cambie después.

6.3.2 Relaciones

Las catorce relaciones uno a muchos se representaron con una clave foránea en la tabla del lado de muchos, y su participación mínima se expresó con `NOT NULL` cuando es obligatoria. Las dos relaciones muchos a muchos, *Receta* e *Incluye*, produjeron las tablas `receta` y `paquete_producto`, cuyas claves combinan las de las dos entidades y que conservan los atributos de la relación, como la cantidad por porción.

La relación ternaria *Acuerda* produjo la tabla `acuerdo`, cuya clave primaria combina las claves del evento, del paquete y del producto, porque ningún par de ellas determina al tercero. Esta tabla no tiene una clave foránea hacia `paquete_producto`: el contenido del paquete es una base y no una lista cerrada, y esa clave haría que el gestor rechazara justamente lo que el negocio permite, acordar en un evento un producto que el paquete no traía.

6.3.3 Entidades débiles

Mesa y Cuenta dependen del evento por identificación, así que sus tablas llevan una clave primaria compuesta por el número de evento y su clave parcial. De esta forma, la mesa tres de un evento y la de otro son filas distintas, y el gestor rechaza una segunda mesa tres en el mismo evento. Su clave foránea hacia el evento borra en cascada, porque una mesa o una cuenta no tienen sentido sin su evento; sin embargo, esa cascada se detiene si el evento ya tiene comandas, pagos o anticipos, cuyas claves impiden el borrado para conservar el historial del dinero.

6.3.4 Jerarquía de Usuario

Para la jerarquía se eligió la estrategia de una tabla para la superclase y una para cada subclase. Aunque las subclases tienen pocos atributos propios, lo que decidió la estrategia fueron sus relaciones: seis relaciones solo admiten a usuarios de una subclase, como la propina, que solo puede ir a un mesero, y eso exige que la clave foránea apunte a la tabla `mesero` y no a `usuario`. Con una sola tabla para toda la jerarquía, el gestor aceptaría una propina para un cocinero; y sin la tabla de la superclase, los datos comunes se repetirían en tres tablas y el nombre de acceso no podría ser único entre todas ellas.

La restricción disjunta se hace cumplir de forma declarativa: la tabla `usuario` guarda un discriminador `rol`, cada subclase lo repite fijo a su valor y apunta a `usuario` con una clave foránea de dos columnas, el identificador y el rol, de modo que un usuario no puede aparecer en dos subclases. La restricción total se verifica con un disparador diferido que, al confirmar la transacción, comprueba que cada usuario tenga su fila en la subclase que corresponde a su rol.

6.4 Esquema resultante

La transformación produjo diecisiete tablas y seis vistas, que se listan en la Tabla 12 junto con el elemento del modelo del que provienen. La Figura 4 muestra el diagrama relacional, en el que se distinguen la clave primaria, las claves foráneas y las columnas generadas de cada tabla.

Tabla	Origen en el modelo extendido
`evento`	Entidad fuerte Evento
`evento_organizador`	Atributo multivaluado de Evento
`usuario`	Superclase Usuario
`mesero`, `bar_cocina`, `cajero_admin`	Subclases de Usuario
`mesa`, `cuenta`	Entidades débiles, con las relaciones *Atiende* y *Tiene*
`producto`	Entidad fuerte Producto, con la relación *Variante*
`ingrediente`, `paquete`	Entidades fuertes
`receta`, `paquete_producto`	Relaciones muchos a muchos *Receta* e *Incluye*
`acuerdo`	Relación ternaria *Acuerda*
`comanda`	Entidad fuerte Comanda, con *Genera*, *Levanta*, *Referencia*, *Prepara* y *Cancela*
`pago`	Entidad fuerte Pago, con *Recibe*, *Maneja* y *Recibe propina*
`anticipo`	Entidad fuerte Anticipo, con *Recibe*

Tabla 12
Tablas del esquema relacional

En total, el esquema tiene noventa y siete columnas y veinticinco claves foráneas. El diccionario de datos, con el tipo, las restricciones y la descripción de cada columna, se encuentra en el repositorio del proyecto.

6.5 Preservación del significado

Una vez ejecutado el esquema, se revisó contra el inventario, elemento por elemento, que conservara el significado del modelo extendido. Las ocho entidades fuertes, las dos débiles y las tres subclases tienen su tabla, y cada relación quedó expresada con restricciones: el máximo de uno, en la tabla donde vive la clave foránea; el máximo de muchos, en que esa columna no es única; y el mínimo, en que la columna sea obligatoria o admita nulos.

Algunas reglas no se hacen cumplir en el esquema, y se dejaron documentadas para que las valide la aplicación. Que todo evento tenga al menos una mesa y todo producto al menos un ingrediente no se exige, porque el evento se registra semanas antes de montar las mesas y la receta se captura después de dar de alta el producto; el número máximo de mesas depende del acomodo del salón; y que el mesero que recibe la propina sea quien atendió la mesa no es una regla, porque un mesero puede cubrir a otro.

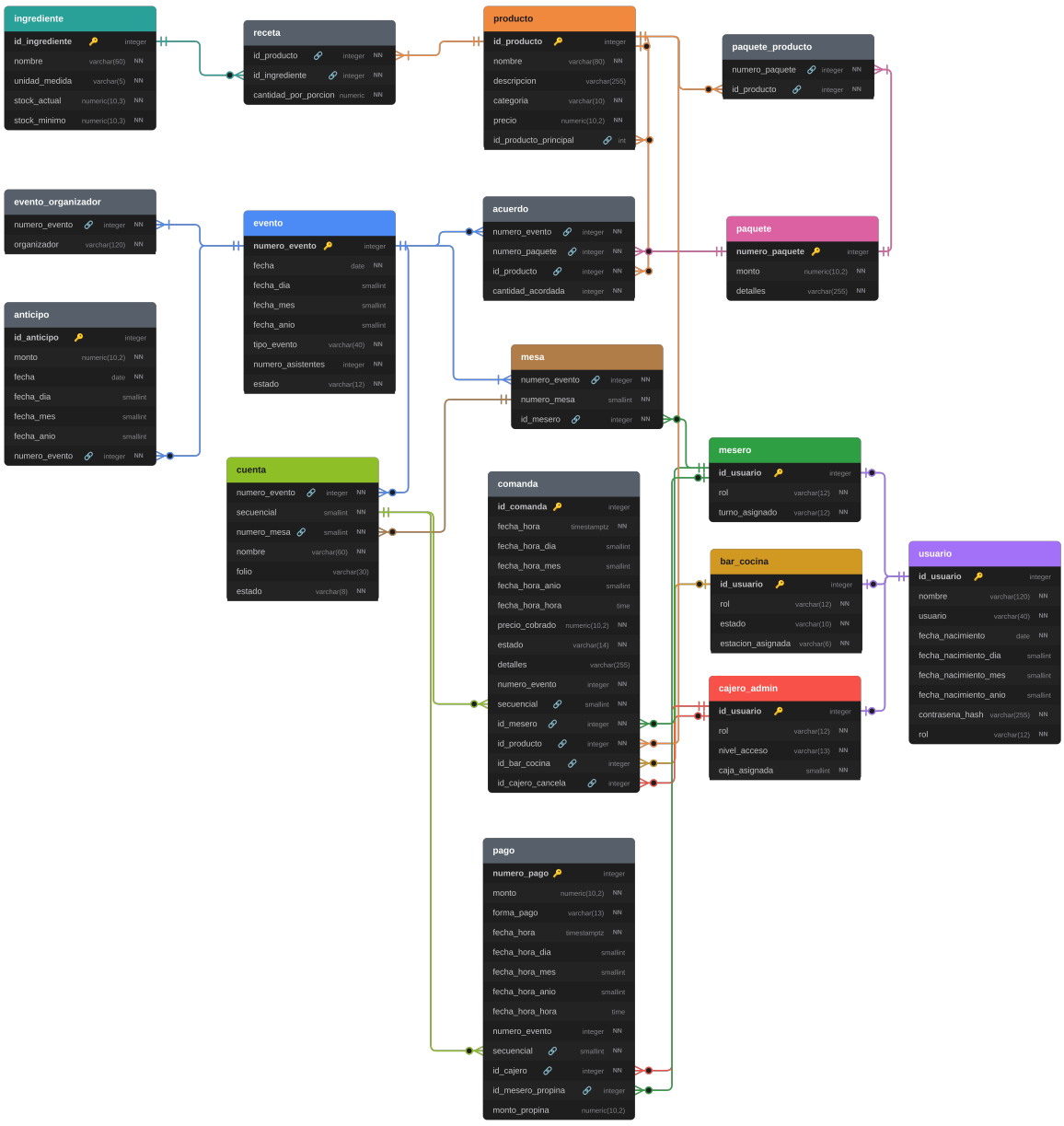


Figura 4
Diagrama relacional del sistema de comandas

La única regla del modelo extendido que no se preservó es el cambio de rol conservando el historial. Como la clave foránea de cada subclase incluye el rol, el gestor impide cambiar el rol de un usuario que ya tiene registros, de modo que el historial queda protegido, pero el cambio de puesto requiere un procedimiento administrativo. Se aceptó esta limitación porque la disyunción es la regla que sostiene el control del dinero y el cambio de rol es un evento poco frecuente; si llegara a ser común, la solución sería separar la identidad de la persona del rol que desempeña en cada periodo.

7 Implementación

En esta sección se describe la implementación física del esquema en PostgreSQL: su ejecución y la forma en que se modificó después, los tipos y dominios, las restricciones de columna, los disparadores y las vistas que hacen cumplir las reglas del negocio, los roles con los que se accede a la base y las pruebas con las que se comprobó cada restricción. Este trabajo se realizó en las prácticas 3 y 4.

7.1 Ejecución del esquema y migraciones

El esquema relacional se escribió en un solo script de definición de datos que crea las diecisiete tablas, sus restricciones, los disparadores, las vistas y un comentario en cada tabla y columna, que el gestor guarda en su catálogo como parte del diccionario de datos. El script se ejecutó sobre una base vacía dentro del contenedor descrito en la metodología, junto con un conjunto pequeño de datos de ejemplo con el que se probaron las restricciones.

A partir de la Práctica 4, el script original ya no se editó: cada cambio posterior al esquema se escribió como una migración numerada con sentencias `ALTER TABLE`, que se aplica después del script y en orden. Así, cualquier base puede llevarse al estado actual ejecutando el script y las migraciones, y la historia de los cambios queda registrada. La Tabla 13 resume las migraciones del proyecto.

Migración	Cambio
`001-dominios.sql`	Crea los dominios `dinero` y `cantidad` y cambia a ellos las columnas de importes y de inventario.
`002-formato-usuario.sql`	Agrega la restricción de formato del nombre de acceso de los usuarios.
`099-permisos.sql`	Crea los roles de la aplicación, de consulta y de administración, y les otorga sus permisos.

Tabla 13
Migraciones aplicadas al esquema

La migración de permisos lleva el número 99 para ejecutarse siempre al final, de modo que alcance a las tablas o columnas que agreguen las migraciones anteriores.

7.2 Tipos de dato y dominios

Cada una de las noventa y siete columnas recibió el tipo y la precisión justificados en el diseño lógico. Ninguna columna usa punto flotante: los importes son decimales exactos a centavos, las cantidades de inventario son decimales exactos a milésimas, las fechas usan los tipos nativos del gestor y los identificadores son enteros que el gestor asigna con una secuencia.

Los importes aparecen en seis columnas de cinco tablas, y las cantidades de inventario en tres columnas de dos tablas. Para no repetir su tipo y su precisión en cada una, se definieron dos dominios con la sentencia `CREATE DOMAIN` (PostgreSQL Global Development Group, s.f.-c), como se muestra en el Código 1. Con ellos, todas las columnas de dinero comparten exactamente la misma definición, y si en el futuro cambiara la precisión, bastaría con modificar el dominio. Las reglas propias de cada columna, como que un precio no sea negativo, se mantuvieron en sus restricciones `CHECK`, porque no son iguales en todas.

```
CREATE DOMAIN dinero AS NUMERIC(10,2);
CREATE DOMAIN cantidad AS NUMERIC(10,3);

ALTER TABLE producto
    ALTER COLUMN precio TYPE dinero USING precio::dinero;
```

Código 1: Dominios de la migración `001-dominios.sql`

7.3 Restricciones de columna

Las restricciones de columna hacen cumplir las reglas que dependen de una sola fila (PostgreSQL Global Development Group, s.f.-a). El esquema declara las siguientes:

- `NOT NULL` en todo atributo sin el cual la fila no tiene sentido y en veintiuna de las veinticinco claves foráneas. Las otras cuatro admiten nulos porque su participación mínima es cero: el producto principal de una variante, la estación que prepara una comanda, el cajero que la canceló y el mesero que recibe la propina.

- `DEFAULT` en trece columnas con un valor inicial natural, como el estado inicial de un evento o de una comanda, la fecha y hora actual o una existencia de cero.

- `UNIQUE` en cuatro columnas o combinaciones, y tres índices únicos para reglas que una restricción simple no expresa: los nombres de acceso y los nombres de ingredientes no pueden repetirse aunque cambien las mayúsculas, y dos cuentas abiertas de una misma mesa no pueden llamarse igual.

- `CHECK` en treinta y seis reglas de rango, de lista de valores, de formato y entre columnas. La Tabla 14 muestra ejemplos de cada tipo.

Tipo	Restricción	Regla del negocio
Rango	`ck_ingrediente_stock`	Las existencias de un ingrediente nunca son negativas.
Lista de valores	`ck_comanda_estado`	Una comanda solo puede estar pendiente, en preparación, lista, entregada o cancelada.
Formato	`ck_usuario_formato`	El nombre de acceso empieza con una letra y solo lleva letras y números, de 3 a 40 caracteres.
Entre columnas	`ck_pago_propina_limite`	La propina no puede superar el monto pagado.
Entre columnas	`ck_comanda_cancelacion`	Una comanda está cancelada si y solo si registra al cajero que la canceló.

Tabla 14
Ejemplos de restricciones `CHECK`

7.4 Reglas del negocio con disparadores

Las reglas que relacionan varias filas o varias tablas no pueden expresarse con una restricción de columna, por lo que se implementaron con disparadores (PostgreSQL Global Development Group, s.f.-i). La Tabla 15 relaciona cada regla del negocio enunciada en el caso de estudio con el disparador que la hace cumplir.

Regla del negocio	Disparador
El precio cobrado se copia del catálogo y no puede modificarse después.	`tg_comanda_precio`
Una comanda solo avanza de estado y una cancelada ya no cambia.	`tg_comanda_estado`
Cada comanda la prepara la estación de su categoría.	`tg_comanda_estacion`
No se levantan comandas en un evento finalizado.	`tg_comanda_evento`
No se levantan comandas en una cuenta cerrada.	`tg_comanda_cuenta_abierta`
Al preparar una comanda se descuenta la receta del inventario.	`tg_comanda_stock`
Un pago no puede exceder el saldo de la cuenta.	`tg_pago_saldo`
Una cuenta solo se cierra sin adeudo y con sus comandas terminadas.	`tg_cuenta_cierre`
Una variante no puede tener variantes.	`tg_producto_variante`
Todo usuario pertenece a la subclase de su rol.	`ct_usuario_total` y uno por subclase

Tabla 15
Reglas del negocio implementadas con disparadores

El descuento de inventario ocurre una sola vez, cuando la comanda pasa por primera vez a preparación, y si algún ingrediente no alcanza, el disparador rechaza la operación con un mensaje que indica cuánto queda y cuánto pide la receta. Los disparadores de la especialización total son diferidos: se revisan al confirmar la transacción, lo que permite insertar primero al usuario y después su fila de subclase.

El Código 2 muestra la función del disparador que protege el cobro. Antes de comparar, blo-

quea la fila de la cuenta con `FOR UPDATE`, de modo que dos cajeros que registren pagos a la misma cuenta al mismo tiempo no puedan exceder el saldo entre los dos: el segundo pago espera a que el primero termine y se compara contra el saldo ya actualizado.

```
CREATE FUNCTION validar_pago_saldo() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
    v_saldo NUMERIC(10,2);
BEGIN
    PERFORM 1 FROM cuenta
    WHERE numero_evento = NEW.numero_evento AND secuencial = NEW.secuencial
    FOR UPDATE;
    SELECT saldo INTO v_saldo FROM v_cuenta
    WHERE numero_evento = NEW.numero_evento AND secuencial = NEW.secuencial;
    IF NEW.monto - COALESCE(NEW.monto_propina, 0) > v_saldo THEN
        RAISE EXCEPTION 'el pago excede el saldo de la cuenta (%)', v_saldo
            USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
END;
$$;
```

Código 2: Función del disparador `tg_pago_saldo`

7.5 Vistas

Los atributos derivados del modelo se calculan en seis vistas (PostgreSQL Global Development Group, s.f.-d), una por cada tabla que los requiere. La más importante para la operación es `v_cuenta`, que calcula para cada cuenta el total acumulado de sus comandas no canceladas, el total pagado sin contar las propinas y el saldo pendiente; tanto el disparador del cobro como el del cierre de cuenta consultan esta vista, de modo que la regla y el reporte usan exactamente el mismo cálculo. Las demás vistas calculan el número de mesas de cada evento, la edad de cada usuario, las propinas acumuladas de cada mesero, el precio vigente del producto de cada comanda y la disponibilidad de cada producto según su receta y el inventario.

7.6 Usuarios y permisos

Una base que siempre se usa con el usuario administrador no está protegida, porque cualquier error de un programa podría borrar o modificar cualquier dato. Por ello se definieron tres roles con el principio de mínimo privilegio (PostgreSQL Global Development Group, s.f.-e; PostgreSQL Global Development Group, s.f.-f), cada uno con un usuario que solo hereda sus permisos y cuya contraseña se toma de variables de entorno. La Tabla 16 los resume.

Rol	Para qué existe y qué puede hacer
comandas_app	Lo usan el servidor del sistema y el script de carga. Opera el salón, pero no puede borrar comandas, eventos, mesas ni cuentas, porque estos cambian de estado en lugar de borrarse; solo puede insertar pagos y anticipos, nunca modificarlos; y no puede alterar la estructura.
comandas_consulta	Para reportes. Solo lee, y no puede ver el *hash* de las contraseñas, porque su permiso sobre la tabla de usuarios se otorgó columna por columna.
comandas_admin	Para correcciones manuales. Puede leer y modificar todas las tablas, pero no es su dueño, así que no puede borrarlas ni crear roles.

Tabla 16
Roles de la base de datos

Los disparadores se ejecutan con los permisos de quien realiza la operación, de modo que levantar una comanda requiere, además del permiso de inserción, poder bloquear la cuenta, consultar el evento y descontar el inventario. Se optó por conceder esos permisos al rol de la aplicación, que de todos modos los necesita para cerrar cuentas, cambiar el estado de los eventos y resurtir ingredientes, en lugar de declarar las funciones con los privilegios de su dueño, lo que no habría aportado ninguna protección.

Cada rol se probó con operaciones permitidas y negadas, y en todos los casos el gestor respondió como se esperaba; por ejemplo, borrar una comanda con el rol de la aplicación produce el error `permission denied for table comanda`. También se comprobó que al retirar un permiso con `REVOKE` la operación deja de funcionar, y que vuelve a funcionar al devolverlo con `GRANT`.

7.7 Pruebas de las restricciones

Cada restricción del esquema se comprobó con una instrucción diseñada para violarla, y cada regla con un caso válido que debe aceptarse. El conjunto de pruebas reúne treinta y seis instrucciones que el gestor debe rechazar y quince casos válidos que debe aceptar, y se ejecuta con un solo comando que se detiene si alguna prueba no da el resultado esperado. Entre ellas, diez casos documentan con detalle las restricciones de dominio: `NOT NULL`, `DEFAULT`, `UNIQUE` simple y compuesta, y `CHECK` de rango, de formato y entre columnas, cada uno con la operación intentada, el mensaje del gestor y la regla protegida. En la última ejecución, las treinta y seis instrucciones fueron rechazadas y los quince casos válidos se aceptaron.

Referencias

Aguilar Vera, R., Naal Jácome, A., Díaz Mendoza, J., & Gómez Gómez, O. (2023). NoSQL database modeling and management: A systematic literature review. *Revista Facultad de Ingeniería*, *32*(65), Artículo e16519. https://doi.org/10.19053/01211129.v32.n65.2023.16519

Atlassian. (s.f.). *Git feature branch workflow*. Atlassian Git Tutorials. Consultado el 16 de septiembre de 2026, desde https://www.atlassian.com/git/tutorials/comparing-workflows/feature-branch-workflow

Bellinger, G., Castro, D., & Mills, A. (2004). *Data, information, knowledge, and wisdom*. Universidade Federal de Minas Gerais (UFMG). Consultado el 18 de septiembre de 2026, desde https://homepages.dcc.ufmg.br/~amendes/SistemasInformacaoTP/TextosBasicos/Data-Information-Knowledge.pdf

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. Consultado el 16 de septiembre de 2026, desde https://git-scm.com/book/en/v2

Chen, P. P.-S. (1976). The entity-relationship model—toward a unified view of data. *ACM Transactions on Database Systems*, *1*(1), 9-36. https://doi.org/10.1145/320434.320440

Date, C. J. (2001). *Introducción a los sistemas de bases de datos* (S. L. M. Ruíz Faudón, Trad.; 7.ª ed.). Pearson Educación.

Docker. (s.f.-a). *Publishing and exposing ports*. Docker Docs. Consultado el 16 de septiembre de 2026, desde https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/

Docker. (s.f.-b). *Volumes*. Docker Docs. Consultado el 16 de septiembre de 2026, desde https://docs.docker.com/engine/storage/volumes/

Docker. (s.f.-c). *What is a container?* Docker Docs. Consultado el 16 de septiembre de 2026, desde https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

GitHub. (s.f.-a). *About pull request reviews*. GitHub Docs. Consultado el 16 de septiembre de 2026, desde https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/about-pull-request-reviews

GitHub. (s.f.-b). *About repositories*. GitHub Docs. Consultado el 16 de septiembre de 2026, desde https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories

Machuca, L., & Braul, R. (2022). Metodologías más usadas en la seguridad de bases de datos: Una revisión de la literatura científica, 2016–2021. *Campus*, *27*(33), 121-130. https://doi.org/10.24265/campus.2022.v27n33.08

PostgreSQL Global Development Group. (s.f.-a). *Constraints*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/ddl-constraints.html

PostgreSQL Global Development Group. (s.f.-b). *COPY*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/sql-copy.html

PostgreSQL Global Development Group. (s.f.-c). *CREATE DOMAIN*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/sql-createdomain.html

PostgreSQL Global Development Group. (s.f.-d). *CREATE VIEW*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/sql-createview.html

PostgreSQL Global Development Group. (s.f.-e). *Database roles*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/database-roles.html

PostgreSQL Global Development Group. (s.f.-f). *Privileges*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/ddl-priv.html

PostgreSQL Global Development Group. (s.f.-g). *SAVEPOINT*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/sql-savepoint.html

PostgreSQL Global Development Group. (s.f.-h). *Transactions*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/tutorial-transactions.html

PostgreSQL Global Development Group. (s.f.-i). *Triggers*. PostgreSQL 16 Documentation. Consultado el 10 de noviembre de 2026, desde https://www.postgresql.org/docs/16/triggers.html

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2002). *Database system concepts: Instructor's manual* (4.ª ed.). McGraw-Hill Higher Education.

Taipalus, T. (2024). Database management system performance comparisons: A systematic literature review. *Journal of Systems and Software*, *208*, Artículo 111872. https://doi.org/10.1016/j.jss.2023.111872